Generative Adversarial Networks

Aprender a generar mediante un juego adversario

Maestría en Inteligencia Artificial

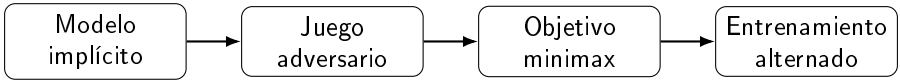
Universidad ORT Uruguay

6 de octubre de 2026

Objetivos de la clase

- Entender la novedad introducida por Goodfellow et al. en 2014.
- Identificar los roles del generador y el discriminador.
- Derivar el objetivo minimax de una GAN.
- Entender el discriminador óptimo y el equilibrio del juego.
- Comparar la loss saturante con la loss no saturante.
- Reconocer problemas prácticos como mode collapse e inestabilidad.

El recorrido de hoy



La pregunta central cambia: de evaluar una densidad a aprender mediante un clasificador.

El trabajo de Goodfellow

Generative Adversarial Nets, 2014

Ian Goodfellow y colaboradores propusieron entrenar un modelo generativo enfrentándolo a otro modelo.

La novedad:

- no hace falta escribir una likelihood explícita y tratable;
- no hace falta una cota variacional;
- la señal de aprendizaje la produce un discriminador;
- el entrenamiento se formula como un juego entre dos redes.

De likelihood explícita a modelo implícito

Hasta ahora buscábamos una distribución evaluable:

$$p_\theta(x) \quad \text{o al menos una cota para} \quad \log p_\theta(x).$$

En una GAN definimos un procedimiento de muestreo:

$$z \sim p_z(z), \qquad x = G_\theta(z).$$

Modelo generativo implícito

Podemos generar muestras de p_g, aunque en general no podamos evaluar su densidad $p_g(x)$.

Qué cambia respecto a máxima verosimilitud

Con máxima verosimilitud:

$$\arg\min_{\theta} \mathrm{KL}(p_{\mathrm{data}} \| p_\theta) = \arg\min_{\theta} \left[-\mathbb{E}_{x \sim p_{\mathrm{data}}} \log p_\theta(x) \right].$$

Esto requiere evaluar $p_\theta(x)$.

Una GAN evita esa evaluación: compara muestras de p_{data} y p_g mediante un clasificador aprendido.

Puente desde VAE

VAE

$$z \sim p(z)$$

$$x \sim p_\theta(x \mid z)$$

- encoder para inferencia;
- decoder probabilístico;
- optimiza una ELBO.

GAN

$$z \sim p_z(z)$$

$$x = G_\theta(z)$$

- generador para muestreo;
- discriminador durante entrenamiento;
- optimiza un juego adversario.

Jugador 1: el generador

$$G_\theta : \mathbb{R}^{d_z} \to \mathbb{R}^{d_x}$$

Recibe ruido simple:

$$z \sim p_z(z), \qquad p_z = \mathcal{N}(0, I) \text{ o } \mathcal{U}(-1, 1),$$

y produce:

$$\tilde{x} = G_\theta(z).$$

La transformación induce una distribución p_g sobre el espacio de datos.

Jugador 2: el discriminador

$$D_\psi(x) \in (0,1)$$

Interpretamos:

$$D_\psi(x) \approx P(y = 1 \mid x),$$

donde:

$$y = \begin{cases} 1, & x \sim p_{\text{data}}, \\ 0, & x \sim p_g. \end{cases}$$

Es un clasificador binario que intenta distinguir datos reales de datos generados.

Los dos conjuntos de entrenamiento

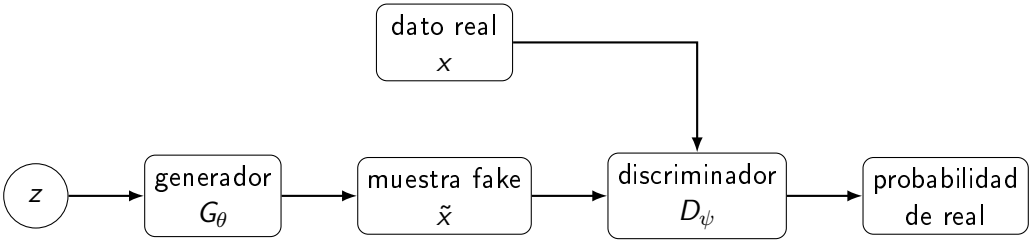
En cada iteración construimos:

$$S_{\text{real}} = \{(x^{(i)}, 1) : x^{(i)} \sim p_{\text{data}}\},$$

$$S_{\text{fake}} = \{(G_\theta(z^{(i)}), 0) : z^{(i)} \sim p_z\}.$$

El dataset negativo cambia porque el generador también está aprendiendo.

Arquitectura completa



Solo el generador queda para producir muestras una vez terminado el entrenamiento.

Repaso: esperanza y promedio muestral

La esperanza de una función es un promedio ponderado por una distribución:

$$\mathbb{E}_{x\sim p}[f(x)] = \int p(x)f(x)\,dx.$$

Si solo tenemos muestras $x^{(1)}, \ldots, x^{(m)} \sim p$, usamos Monte Carlo:

$$\mathbb{E}_{x\sim p}[f(x)] \approx \frac{1}{m}\sum_{i=1}^{m} f(x^{(i)}).$$

En entrenamiento, esas muestras forman un minibatch.

Repaso: logaritmo y binary cross-entropy

Propiedades que usaremos:

$$\log(ab) = \log a + \log b, \qquad \log(1) = 0, \qquad \log(u) \to -\infty \text{ si } u \to 0^+.$$

Para una etiqueta $y \in \{0, 1\}$ y una probabilidad D:

$$\mathrm{BCE}(y, D) = -y \log D - (1 - y) \log(1 - D).$$

Minimizar BCE premia $D \to 1$ cuando $y = 1$ y $D \to 0$ cuando $y = 0$.

La loss del discriminador

El discriminador es un clasificador binario. Maximiza:

$$\mathbb{E}_{x\sim p_{\text{data}}}\left[\log D_\psi(x)\right] + \mathbb{E}_{z\sim p_z}\left[\log\left(1 - D_\psi(G_\theta(z))\right)\right].$$

Equivale a minimizar binary cross-entropy:

$$\mathcal{L}_D = -\mathbb{E}_{p_{\text{data}}} \log D_\psi(x) - \mathbb{E}_{p_z} \log(1 - D_\psi(G_\theta(z))).$$

Objetivos opuestos

Discriminador

- quiere $D(x) \to 1$ para datos reales;
- quiere $D(G(z)) \to 0$ para datos falsos.

Generador

- quiere producir muestras realistas;
- quiere $D(G(z)) \to 1$;
- aprende a engañar al discriminador.

Aprendizaje adversario

El discriminador mejora la señal; el generador aprende a vencer esa señal.

Repaso: mínimo, máximo y juego de suma cero

$$\min_{\theta} F(\theta)$$

busca parámetros que hagan pequeño a F, mientras que:

$$\max_{\psi} F(\psi)$$

busca parámetros que lo hagan grande.

En:

$$\min_{\theta} \max_{\psi} V(\theta, \psi),$$

primero leemos el máximo para θ fijo y luego el mínimo exterior.

Si un jugador gana exactamente lo que el otro pierde, el juego es de suma cero.

Objetivo original

Goodfellow et al. proponen:

$$\boxed{\min_{\theta} \max_{\psi} V(D_\psi, G_\theta)}$$

$$V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))] .$$

El discriminador maximiza V; el generador intenta minimizarlo.

En esta versión es un juego de suma cero: ambos optimizan el mismo valor en direcciones opuestas.

Quién controla cada término

$$\underbrace{\mathbb{E}_{x\sim p_{\text{data}}} \log D_\psi(x)}_{\text{solo depende de } \psi} + \underbrace{\mathbb{E}_{z\sim p_z} \log(1 - D_\psi(G_\theta(z)))}_{\text{depende de } \psi \text{ y } \theta}$$

- Al actualizar D, G queda congelado.
- Al actualizar G, D queda congelado.
- El gradiente para G atraviesa D, aunque no actualice sus pesos.

No resolvemos el minimax de una vez

Idealmente:

$$\min_{\theta} \left[\max_{\psi} V(D_\psi, G_\theta) \right].$$

En práctica alternamos:

1. uno o más pasos para mejorar D_ψ;
2. un paso para mejorar G_θ;
3. repetir con nuevos datos y nuevo ruido.

Cada jugador persigue un objetivo que cambia mientras el otro aprende.

Repaso: clasificador bayesiano

Para dos clases, Bayes dice:

$$P(y = 1 \mid x) = \frac{p(x \mid y = 1)P(y = 1)}{p(x \mid y = 1)P(y = 1) + p(x \mid y = 0)P(y = 0)}.$$

En GAN:

$$p(x \mid y = 1) = p_{\text{data}}(x), \qquad p(x \mid y = 0) = p_g(x).$$

Si ambas clases tienen prior $1/2$, esos factores se cancelan.

Discriminador óptimo: intuición

Supongamos que G está fijo.

Para cada x, el discriminador observa dos fuentes:

$$x \sim p_{\text{data}} \qquad \text{o} \qquad x \sim p_g.$$

Si ambas clases tienen igual prior, el clasificador bayesiano responde:

$$D^\star(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

Repaso: derivadas que vamos a usar

La derivada mide la pendiente local de una función.

Reglas necesarias:

$$\frac{d}{du}\log u = \frac{1}{u},$$

$$\frac{d}{du}\log(1-u) = \frac{1}{1-u}\frac{d(1-u)}{du} = -\frac{1}{1-u}.$$

Además, las constantes salen de la derivada:

$$\frac{d}{du}\left[a\, f(u)\right] = a\, f'(u).$$

Repaso: cómo verificar un máximo

Un punto interior $u^\star$ puede ser candidato a máximo si:

$$f'(u^\star) = 0.$$

Si además:

$$f''(u) < 0,$$

la función es cóncava y ese punto es un máximo.

Para:

$$f(D) = a \log D + b \log(1 - D),$$

tenemos:

$$f''(D) = -\frac{a}{D^2} - \frac{b}{(1-D)^2} < 0 \quad \text{si } a, b > 0.$$

Discriminador óptimo: derivación

Para un x fijo, maximizamos:

$$f(D) = p_{\text{data}}(x) \log D + p_g(x) \log(1 - D).$$

Aplicamos las reglas del repaso:

$$f'(D) = p_{\text{data}}(x) \frac{1}{D} + p_g(x) \left(-\frac{1}{1 - D} \right).$$

Como f es cóncava, igualamos $f'(D) = 0$ y despejamos:

$$\boxed{D^\star(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}}.$$

El equilibrio ideal

Si el generador logra:

$$p_g = p_{\text{data}},$$

entonces:

$$D^\star(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_{\text{data}}(x)} = \frac{1}{2}.$$

El discriminador no sabe distinguir porque ambas fuentes tienen la misma distribución.

Que $D(x) = 1/2$ es deseable solo al final del juego, no al comienzo.

Repaso: por qué no alcanza una sola KL

Recordemos que:

$$\mathrm{KL}(p\|q) = \mathbb{E}_{x\sim p}\left[\log \frac{p(x)}{q(x)}\right].$$

- Tiene dirección: $\mathrm{KL}(p\|q) \neq \mathrm{KL}(q\|p)$.
- Pregunta cuánto perdemos al representar muestras de p usando q.
- Si $p(x) > 0$ y $q(x) = 0$, el cociente produce una divergencia infinita.

Para comparar dos distribuciones como pares, queremos una construcción simétrica.

Jensen–Shannon: crear un punto de encuentro

Primero construimos una distribución intermedia:

$$m(x) = \frac{1}{2}p(x) + \frac{1}{2}q(x).$$

Muestrear de m equivale a:

1. lanzar una moneda justa;
2. si sale cara, muestrear de p;
3. si sale cruz, muestrear de q.

m contiene la mitad de cada distribución.

Jensen–Shannon: comparar con la mezcla

Ahora medimos:

$$\mathrm{KL}(p\|m) \qquad \text{y} \qquad \mathrm{KL}(q\|m).$$

Jensen–Shannon es el promedio de esas dos comparaciones:

$$\mathrm{JS}(p\|q) = \frac{1}{2}\mathrm{KL}(p\|m) + \frac{1}{2}\mathrm{KL}(q\|m).$$

- Es simétrica: intercambiar p y q no cambia el resultado.
- Vale cero exactamente cuando $p = q$.
- Con logaritmo natural: $0 \leq \mathrm{JS}(p\|q) \leq \log 2$.

Jensen–Shannon: ejemplo extremo

Consideremos dos resultados posibles, A y B:

$$p = (1, 0), \qquad q = (0, 1).$$

No comparten soporte, pero la mezcla sí cubre ambos:

$$m = \frac{1}{2}(p + q) = \left(\frac{1}{2}, \frac{1}{2}\right).$$

Entonces:

$$\mathrm{KL}(p\|m) = \log 2, \qquad \mathrm{KL}(q\|m) = \log 2,$$

$$\mathrm{JS}(p\|q) = \frac{1}{2}\log 2 + \frac{1}{2}\log 2 = \log 2.$$

Distribuciones totalmente separadas alcanzan la JS máxima.

Conexión con Jensen–Shannon

Al reemplazar el discriminador óptimo en el valor del juego:

$$V(D^\star, G) = -\log 4 + 2\,\mathrm{JS}(p_{\mathrm{data}} \,\|\, p_g)\,.$$

Como:

$$\mathrm{JS}(p_{\mathrm{data}} \| p_g) \geq 0,$$

el mínimo global ocurre cuando:

$$p_g = p_{\mathrm{data}}, \qquad V(D^\star, G) = -\log 4.$$

La GAN original induce una comparación entre distribuciones sin evaluar explícitamente $p_g(x)$.

Qué demuestra la teoría y qué no

La teoría demuestra:

- cuál es $D^\star$ para un generador fijo;
- cuál es el equilibrio global en el espacio de distribuciones.

No garantiza:

- que redes finitas alcancen ese equilibrio;
- que SGD converja en un juego no convexo;
- que ambos jugadores avancen a velocidades compatibles.

Repaso: gradiente y descenso

Si una loss depende de muchos parámetros:

$$\nabla_\theta \mathcal{L} = \left(\frac{\partial \mathcal{L}}{\partial \theta_1}, \ldots, \frac{\partial \mathcal{L}}{\partial \theta_n} \right),$$

el gradiente señala la dirección de mayor crecimiento.

Para minimizar, damos un paso en la dirección opuesta:

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}.$$

Backpropagation aplica la regla de la cadena para calcular este gradiente a través de toda la red.

Paso del discriminador

Muestreamos:

$$x^{(i)} \sim p_{\text{data}}, \qquad z^{(i)} \sim p_z, \qquad \tilde{x}^{(i)} = G_\theta(z^{(i)}).$$

Con G congelado, minimizamos:

$$\mathcal{L}_D = -\frac{1}{m} \sum_{i=1}^{m} \left[\log D_\psi(x^{(i)}) + \log(1 - D_\psi(\tilde{x}^{(i)})) \right].$$

Loss minimax del generador

El objetivo original pide minimizar:

$$\mathcal{L}_G^{\text{sat}} = \mathbb{E}_{z \sim p_z} \left[\log(1 - D_\psi(G_\theta(z))) \right].$$

Pero al comienzo D puede reconocer fácilmente las muestras falsas:

$$D(G(z)) \approx 0.$$

En esa región, esta loss puede entregar un gradiente muy pequeño al generador.

Repaso: sigmoide y regla de la cadena

La sigmoide transforma un logit $a \in \mathbb{R}$ en una probabilidad:

$$D = \sigma(a) = \frac{1}{1 + e^{-a}}.$$

Su derivada es:

$$\frac{dD}{da} = D(1 - D).$$

Para una composición $L(D(a))$, la regla de la cadena dice:

$$\frac{dL}{da} = \frac{dL}{dD}\frac{dD}{da}.$$

Esto conecta la pendiente de la loss con el logit producido por la red.

Dónde se satura el gradiente

Sea a el logit del discriminador:

$$D(G(z)) = \text{sigmoid}(a).$$

Para la loss minimax:

$$\frac{\partial}{\partial a} \log(1 - D) = -\frac{1}{1 - D} D(1 - D) = -D.$$

Si el discriminador está muy seguro de que la muestra es falsa:

$$D \approx 0 \quad \Longrightarrow \quad \frac{\partial \mathcal{L}_G^{\text{sat}}}{\partial a} \approx 0.$$

El generador recibe poca señal justo cuando más la necesita.

Loss no saturante

En práctica entrenamos al generador maximizando:

$$\mathbb{E}_{z\sim p_z}\left[\log D_\psi(G_\theta(z))\right],$$

o equivalentemente minimizando:

$$\boxed{\mathcal{L}_G^{\mathrm{NS}} = -\mathbb{E}_{z\sim p_z}\left[\log D_\psi(G_\theta(z))\right]}.$$

Tiene el mismo punto fijo deseado, pero suele producir mejores gradientes.

Por qué la no saturante ayuda

Para:

$$\mathcal{L}_G^{\mathrm{NS}} = -\log D,$$

la derivada respecto del logit es:

$$\frac{\partial \mathcal{L}_G^{\mathrm{NS}}}{\partial a} = -\frac{1}{D}\, D(1-D) = D - 1.$$

Si $D \approx 0$:

$$D - 1 \approx -1.$$

La señal sigue siendo grande cuando el discriminador rechaza con seguridad lo generado.

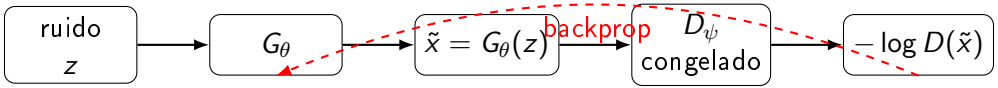
Dos losses, un mismo objetivo conceptual

Loss de G	Fórmula	Si $D(G(z)) \approx 0$
Minimax	$\log(1 - D(G(z)))$	gradiente débil
No saturante	$-\log D(G(z))$	gradiente fuerte

La versión no saturante ya no es literalmente el mismo juego de suma cero, pero conserva el equilibrio:

$$p_g = p_{\text{data}}.$$

El camino del gradiente para G



Congelar D significa no actualizar ψ; no significa cortar la derivada de D respecto de su entrada.

Algoritmo de entrenamiento

Para cada iteración:

1. Muestrear un minibatch real $x \sim p_{\text{data}}$.
2. Muestrear ruido $z \sim p_z$ y generar $\tilde{x} = G_\theta(z)$.
3. Actualizar ψ minimizando $\mathcal{L}_D$, con $\tilde{x}$ separado del grafo de G.
4. Muestrear nuevo ruido $z \sim p_z$.
5. Congelar los pesos de D.
6. Actualizar θ minimizando $\mathcal{L}_G^{\text{NS}}$.

Usar ruido nuevo evita entrenar G contra un único lote de ejemplos falsos.

Implementación numéricamente estable

En código, conviene que D produzca un logit:

$$a_\psi(x) \in \mathbb{R}, \qquad D_\psi(x) = \text{sigmoid}(a_\psi(x)).$$

Luego usamos una operación combinada:

$$\texttt{BCEWithLogits}(a, y).$$

- evita calcular $\log(\text{sigmoid}(a))$ manualmente;
- reduce problemas de overflow y $\log(0)$;
- produce el mismo objetivo con mayor estabilidad numérica.

Dinámica intuitiva

1. G produce muestras fáciles de detectar.
2. D aprende qué rasgos las delatan.
3. El gradiente de D indica a G cómo cambiar sus muestras.
4. G mejora y obliga a D a encontrar nuevos errores.

Dificultad

No optimizamos una loss fija: cada red modifica el paisaje de optimización de la otra.

Mode collapse

El generador puede descubrir unas pocas muestras que engañan a D:

$$z_1, z_2, \ldots, z_n \quad \longmapsto \quad G(z_1) \approx G(z_2) \approx \cdots \approx x^\star.$$

Resultado:

- muestras individuales convincentes;
- poca diversidad;
- partes de p_{data} nunca representadas.

Calidad alta no implica cobertura alta.

Oscilación e inestabilidad

En una optimización usual buscamos un mínimo. En una GAN buscamos un equilibrio entre dos jugadores.

Puede ocurrir que:

- G persiga continuamente las debilidades actuales de D;
- D olvide estrategias anteriores;
- las losses oscilen sin converger;
- pequeños cambios de hiperparámetros alteren el resultado.

Cuando un jugador domina

D demasiado fuerte

- separa perfectamente;
- puede dar gradientes poco informativos;
- G deja de mejorar.

G explota a D

- encuentra pocos patrones efectivos;
- cae en mode collapse;
- D debe recuperar terreno.

El objetivo práctico no es maximizar la accuracy de D, sino sostener una señal útil para G.

Prácticas de estabilización

- Usar la loss no saturante para G.
- Normalizar los datos al rango esperado por la salida de G.
- Usar arquitecturas adecuadas: convoluciones, LeakyReLU y normalización.
- Ajustar por separado learning rates y cantidad de pasos de D y G.
- Evitar que D memorice el dataset.
- Guardar muestras para un conjunto fijo de vectores z.
- Comparar calidad y diversidad, no solo losses.

Son heurísticas: no reemplazan una formulación con mejores propiedades.

Las losses no cuentan toda la historia

En una GAN:

- una loss de D baja puede indicar que G es malo;
- una loss de D cercana al azar puede indicar equilibrio o un D débil;
- la loss de G depende del adversario actual;
- no tenemos una log-likelihood directa para comparar modelos.

Consecuencia

No existe una curva única cuya caída garantice mejores muestras.

Cómo evaluar una GAN

- **Muestras:** ¿cada ejemplo parece real?
- **Diversidad:** ¿aparecen distintos modos del dataset?
- **Interpolaciones:** ¿el espacio latente cambia de forma coherente?
- **Conjunto fijo de z:** ¿cómo evoluciona la misma semilla?
- **Precision/recall generativo:** calidad vs cobertura.
- **FID:** distancia entre representaciones de datos reales y generados.

Ninguna métrica reemplaza revisar qué aspectos del problema realmente importan.

GAN y VAE: sesgos distintos

	VAE	GAN
Objetivo	ELBO	juego adversario
Inferencia $x \to z$	sí, con encoder	no en GAN básica
Densidad evaluable	aproximada	no
Muestras	a veces suaves	frecuentemente nítidas
Entrenamiento	una loss conjunta	dos redes en competencia
Riesgo típico	posterior collapse	mode collapse

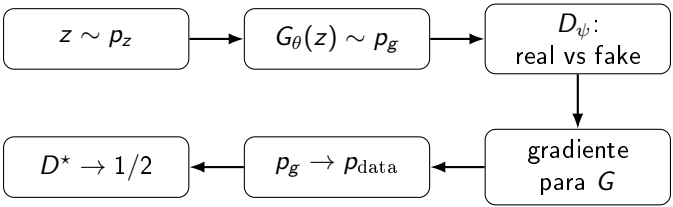
Más allá de la GAN original

La formulación original compara distribuciones a través de un clasificador. Otras variantes cambian la discrepancia o la arquitectura:

- **DCGAN:** pautas convolucionales para imágenes.
- **Conditional GAN:** generación condicionada por una etiqueta o entrada.
- **WGAN:** reemplaza el discriminador por un crítico y usa distancia Wasserstein.
- **WGAN-GP:** impone la restricción Lipschitz mediante gradient penalty.

Estas variantes responden, en gran parte, a los problemas de estabilidad y control.

El mapa completo



Resumen

- Una GAN aprende una distribución implícita transformando ruido.
- El discriminador aprende a separar datos reales de generados.
- El generador aprende a engañar al discriminador.
- Con G fijo, $D^\star(x) = p_{\text{data}}(x)/(p_{\text{data}}(x) + p_g(x))$.
- En el equilibrio ideal, $p_g = p_{\text{data}}$ y $D^\star = 1/2$.
- La loss no saturante mejora la señal inicial para G.
- El entrenamiento puede sufrir mode collapse e inestabilidad.

Para llevar

Idea central

Una GAN reemplaza una función de pérdida generativa explícita por un adversario aprendido que indica en qué se diferencian las muestras falsas de las reales.

$$\min_G \max_D \mathbb{E}_{x\sim p_{\text{data}}} \log D(x) + \mathbb{E}_{z\sim p_z} \log(1 - D(G(z))).$$

Preguntas para chequear

1. ¿Por qué una GAN es un modelo generativo implícito?
2. ¿Qué datos usa el discriminador y qué etiqueta recibe cada uno?
3. ¿Por qué el entrenamiento debe ser alternado?
4. ¿Cuál es el discriminador óptimo para un generador fijo?
5. ¿Por qué se usa la loss no saturante?
6. ¿Qué es mode collapse y por qué una muestra aislada no lo revela?

Bibliografía sugerida

- Ian Goodfellow et al. *Generative Adversarial Nets*. NeurIPS, 2014.
- Ian Goodfellow. *NIPS 2016 Tutorial: Generative Adversarial Networks*. arXiv, 2016.
- Alec Radford, Luke Metz y Soumith Chintala. *Unsupervised Representation Learning with Deep Convolutional GANs*. ICLR, 2016.
- Martin Arjovsky, Soumith Chintala y Léon Bottou. *Wasserstein GAN*. ICML, 2017.
- Ishaan Gulrajani et al. *Improved Training of Wasserstein GANs*. NeurIPS, 2017.
- Kevin P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2023.